# 1. vanilla-ice 🥶

*What You See Is What You Get (WYSIWIG).*


- 1. vanilla-ice 🥶
  - 1.1. HOWTO, WTF, RTFM
    - 1.1.1. Prerequisites
    - 1.1.2. Build
    - 1.1.3. Third-party
  - 1.2. Design
    - 1.2.1. gena (GENerate Alerts)
      - 1.2.1.1. ar10 (Apply Rule of 10 sec)
    - 1.2.2. ice (Integrity Check Events)
    - 1.2.3. sender
    - 1.2.4. colya (COLlector Yet Another)
  - 1.3. Extras
    - 1.3.1. Won't be done


## 1.1. HOWTO, WTF, RTFM

### 1.1.1. Prerequisites

- GCC/Clang with common core C++23 libs;
- CMake 3.14+;
- OpenSSL (runtime lib + headers) installed to satisfy `find_package(OpenSSL REQUIRED)`;

**Optional**: Use Python 3.8+ for input data generation (`tools/gen-baseline.py`, `tools/gen-events.py`). Also do not forget about executable bit (`set +x`).

### 1.1.2. Build

In project directory

```
make build
```

will produce `vanilla-ice/build` executable artifacts:

1. `gena`
2. `ice`
3. `sender`
4. `colya`

For getting help, just type `make` or `make help`.

```
[user@pod sr60]$ make
------------------------------------------------------------------
                          Vanilla Ice
------------------------------------------------------------------
build-test            build test [`test-runner`]
build-vanilla         build demo executables for `vanilla-ice`
clean                 remove build artifacts
run-test              run `test-runner`
[user@pod sr60]$
```

For cleaning up build artifacts, type `make clean`.

**Example:** 1. clean artifacts, 2. build unit tests, 3. run unit tests, 4. build `vanilla-ice/build` executables.

```
make clean build-test run-test build-vanilla
```

### 1.1.3 Third-party

1. OpenSSL
2. Niels Lohmann's JSON
3. Yuji Hirose's cpp-httplib
4. Google Test

## 1.2. Design

### 1.2.1. gena (GENerate Alerts)

Piece of software according to functional requirements:

- get events from file (`events.log`) in JSON Lines format;
- for `ProcessStarted` events apply rule, if match → `SuspiciousActivityAlert`;

Getting events from file:

- `sensor` aka `producer` multi-thread read `events.log` → push to `queue`;
- `queue` w/o busy-wait (use condition_variable), back pressure (desired);
- `exam` aka `consumer` multi-thread → pop from `queue`, send to capacitor queue, arbiter apply rule → send alerts;
- grace termination (Ctrl+C);

**Note:** it's better to use **absolute paths** in args.

Overview:

```
[IN]
arg1 - file path (JSON-lines events);

[OUT]
```

```
stdout - JSON lines (alerts) etc.

{"alert":"SuspiciousActivityAlert","ppid":1087","pids":
[7093,9678,6964,6403,2699,8612]}
...
```

Usage example:

```
build/gena tools/events.log | build/sender 127.0.0.3 3333
```

#### 1.2.1.1. AR-10 (Apply Rule of 10 seconds)

If a process (from `ProcessStarted` event type) has **spawned > 5 children within 10 seconds** → alert `SuspiciousActivityAlert`.

Alert must contain:

- PID of suspicious (aka "fertile" parent) proccess // PPID;
- list of spawned children PIDs // PIDs

Also source code should demonstrate usage of **sliding window** based on **timestamps** in multi-threaded environment.

### 1.2.2. ice (Integrity Check Events)

Piece of software according to functional requirements:

- calculate hashsum (SHA-256) for files in given dir;
- compare with reference (from `baseline.json`), if mismatch → event op: `file_added` / `file_modified` / `file_removed`;

**Note:** it's better to use **absolute paths** in args.

Overview:

```
[IN]
arg1 - dir path;
arg2 - baseline manifest (valid JSON);

[OUT]
stdout - JSON lines (events) etc.

{"op":"file_added","path":"/home/user/ida/loaders/new_file.odt"}
{"op":"file_modified","path":"/home/user/ida/themes/default/theme.css"}
{"op":"file_removed","path":"/home/user/ida/eicar.com"}
...
```

Usage example:

```
build/ice ~/ida  ~/baseline-for-ida.json | build/sender 127.0.0.3 3333
```

### 1.2.3. sender

Read `stdin` (JSON-lines format is expected), then send lines to collector `colya` at TCP `host:port`.

Overview:

```
[IN]
arg1 - host (string);
arg2 - port (number);

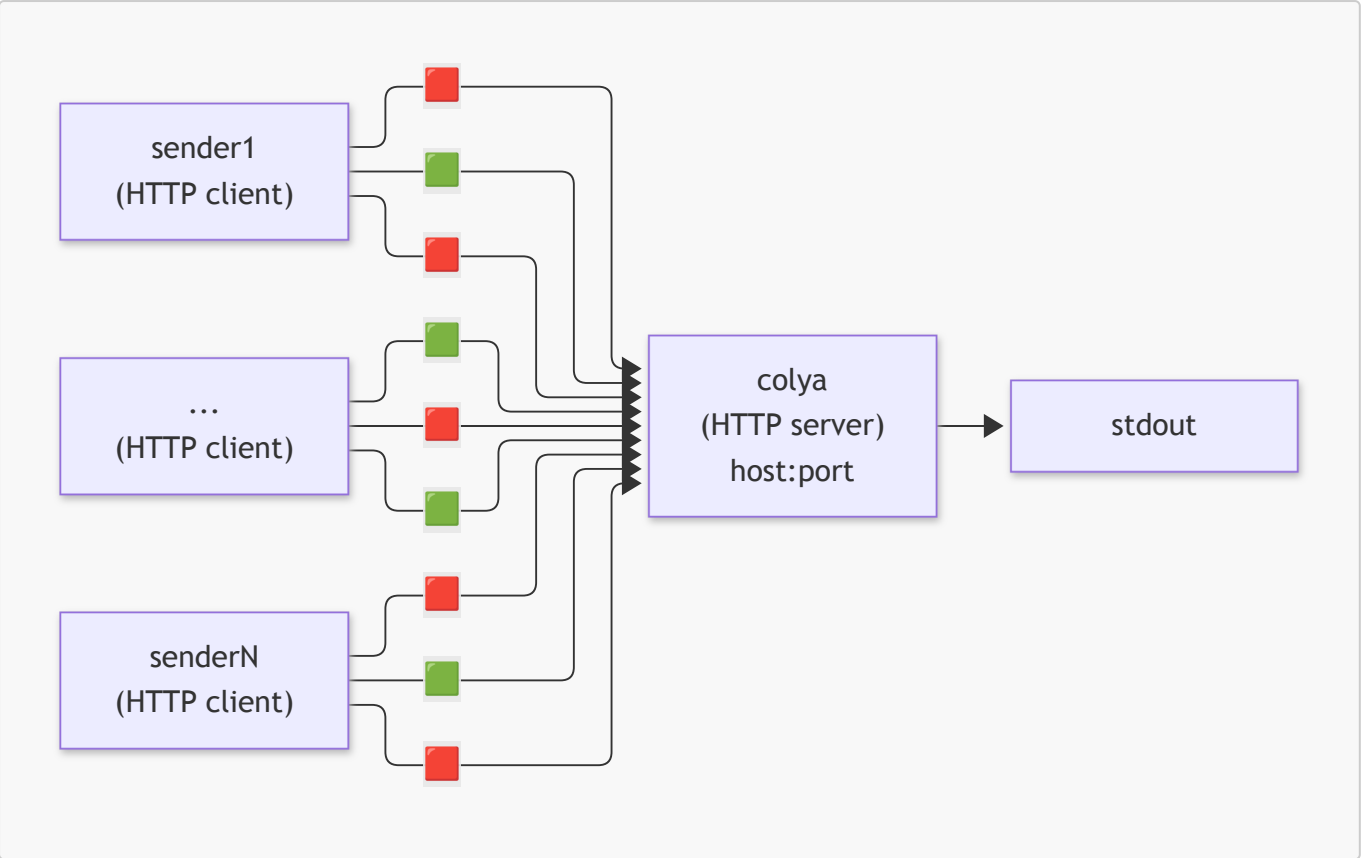
[OUT]
stdout - "Sent: ...", "Error: ...", "Not sent: ..." etc.
```

Usage example:

```
build/gena | build/sender 127.0.0.3 3333
build/ice ... | build/sender 127.0.0.3 3333
```

### 1.2.4. colya ("COLlector Yet Another")

From any number of clients (e.g. `gena | sender` or `ice | sender`) at `host:port` gets POST-requests, then reads JSON-lines of alerts/events and print them to `stdout` *as is*.




Overview:

```
[IN]
arg1 - host (string);
arg2 - port (number);

[OUT]
stdout - JSON-lines

{"op":"file_added","path":"/home/user/ida/loaders/new_file.odt"}
{"op":"file_removed","path":"/home/user/ida/eicar.com"}
{"alert":"SuspiciousActivityAlert","ppid":1087","pids":
[7093,9678,6964,6403,8612,2699]}
{"op":"file_modified","path":"/home/user/ida/themes/default/theme.css"}
```

Usage example:

```
build/colya 127.0.0.3 3333
```

# 1.3. Extras

## 1.3.1. Won't be done

- Real monitoring via API OS (e.g. `ProcessStarted`, `FileWritten` etc.);
- Cross-compiling (including MSVC), Dockerfile (Containerfile), CI/CD and so on;

- Persistency, TOML/YAML config files, daemon, systemd unit-file;
- Proper exception, error, and corner case handling;
- Proper testing;
- Installer/package;